

iOS面试题合集（组件化+性能优化篇）

iOSer iOSer 5月19日

收录于话题

#BAT大厂面试题合集

9个

欢迎点击上方蓝字“iOSer”关注我们
再点击右上角“...”菜单，选择“设为星标”
我们一起精进、成长！

本栏目将持续更新--请iOS的小伙伴关注我们!

做这个的初心是希望能巩固自己的基础知识，

当然也希望能帮助更多的开发者，

可以加入我们的技术交流群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

目录：

一，组件化

- 1.组件化有什么好处？
- 2.你是如何组件化解耦的？
- 3.为什么CTMediator方案优于基于Router的方案？
4. 基于CTMediator的组件化方案，有哪些核心组成？

二，性能优化

1. 造成tableView卡顿的原因有哪些？

- 2.如何提升 tableview 的流畅度？
3. APP启动时间应从哪些方面优化？
- 4.如何降低APP包的大小
- 5.如何检测离屏渲染与优化
- 6.怎么检测图层混合
- 7.日常如何检查内存泄露？
- 8.如何优化 APP 的电量？

一，组件化

1.组件化有什么好处？

- 业务分层、解耦，使代码变得可维护；
- 有效的拆分、组织日益庞大的工程代码，使工程目录变得可维护；
- 便于各业务功能拆分、抽离，实现真正的功能复用；
- 业务隔离，跨团队开发代码控制和版本风险控制的实现；
- 模块化对代码的封装性、合理性都有一定的要求，提升开发同学的设计能力；
- 在维护好各级组件的情况下，随意组合满足不同客户需求；（只需要将之前的多个业务组件模块在新的主App中进行组装即可快速迭代出下一个全新App）

2.你是如何组件化解耦的？

- 分层

基础功能组件：按功能分库，不涉及产品业务需求，跟库Library类似，通过良好的接口拱上层业务组件调用；不写入产品定制逻辑，通过扩展接口完成定制；

基础UI组件：各个业务模块依赖使用，但需要保持好定制扩展的设计

业务组件：业务功能间相对独立，相互间没有Model共享的依赖；业务之间的页面调用只能通过UIBus进行跳转；业务之间的逻辑Action调用只能通过服务提供；

- 中间件：target-action, url-block, protocol-class

3.为什么CTMediator方案优于基于Router的方案？

Router的缺点：

- 在组件化的实施过程中，注册URL并不是充分必要条件。组件是不需要向组件管理器注册URL的，注册了URL之后，会造成不必要的内存常驻。
- 注册URL的目的其实是一个服务发现的过程，在iOS领域中，服务发现的方式是不需要通过主动注册的，使用runtime就可以了。
- 另外，注册部分的代码的维护是一个相对麻烦的事情，每一次支持新调用时，都要去维护一次注册列表。如果有调用被弃用了，是经常会忘记删项目的。runtime由于不存在注册过程，那就也不会产生维护的操作，维护成本就降低了。
- 由于通过runtime做到了服务的自动发现，拓展调用接口的任务就仅在于各自的模块，任何一次新接口添加，新业务添加，都不必去主工程做操作，十分透明。
- 在iOS领域里，一定是组件化的中间件为openURL提供服务，而不是openURL方式为组件化提供服务。
- 如果在给App实施组件化方案的过程中是基于openURL的方案的话，有一个致命缺陷：非常规对象(不能被字符串化到URL中的对象，例如UIImage)无法参与本地组件间调度。
- 在本地调用中使用URL的方式其实是不必要的，如果业务工程师在本地间调度时需要给出URL，那么就不可避免要提供params，在调用时要提供哪些params是业务工程师很容易懵逼的地方。

- 为了支持传递非常规参数，蘑菇街的方案采用了protocol，这个会侵入业务。由于业务中的某个对象需要被调用，因此必须要符合某个可被调用的protocol，然而这个protocol又不存在于当前业务领域，于是当前业务就不得不依赖public Protocol。这对于将来的业务迁移是有非常大的影响的。

CTMediator的优点：

- 调用时，区分了本地应用调用和远程应用调用。本地应用调用为远程应用调用提供服务。
- 组件仅通过Action暴露可调用接口，模块与模块之间的接口被固化在了Target-Action这一层，避免了实施组件化的改造过程中，对Business的侵入，同时也提高了组件化接口的可维护性。
- 方便传递各种类型的参数。

4.基于CTMediator的组件化方案，有哪些核心组成？

- CTMediator中间件：集成就可以了
- 模块Target_%@：模块的实现及提供对外的方法调用Action_methodName，需要传参数时，都统一以NSDictionary*的形式传入。
- CTMediator+%@扩展：扩展里声明了模块业务的对外接口，参数明确，这样外部调用者可以很容易理解如何调用接口。

二，性能优化篇

1.造成tableView卡顿的原因有哪些？

- 1.最常用的就是cell的重用， 注册重用标识符

如果不重用cell时，每当一个cell显示到屏幕上时，就会重新创建一个新的cell

如果有很多数据的时候，就会堆积很多cell。

如果重用cell，为cell创建一个ID，每当需要显示cell 的时候，都会先去缓冲池中寻找可循环利用的cell，如果没有再重新创建cell

- 2.避免cell的重新布局

cell的布局填充等操作 比较耗时，一般创建时就布局好

如可以将cell单独放到一个自定义类，初始化时就布局好

- 3.提前计算并缓存cell的属性及内容

当我们创建cell的数据源方法时，编译器并不是先创建cell 再定cell的高度

而是先根据内容一次确定每一个cell的高度，高度确定后，再创建要显示的cell，滚动时，每当cell进入凭虚都会计算高度，提前估算高度告诉编译器，编译器知道高度后，紧接着就会创建cell，这时再调用高度的具体计算方法，这样可以方式浪费时间去计算显示以外的cell

- 4.减少cell中控件的数量

尽量使cell得布局大致相同，不同风格的cell可以使用不用的重用标识符，初始化时添加控件，

不适用的可以先隐藏

- 5.不要使用ClearColor，无背景色，透明度也不要设置为0

渲染耗时比较长

- 6.使用局部更新

如果只是更新某组的话，使用reloadSection进行局部更

- 7.加载网络数据，下载图片，使用异步加载，并缓存
- 8.少使用addView 给cell动态添加view
- 9.按需加载cell，cell滚动很快时，只加载范围内的cell
- 10.不要实现无用的代理方法，tableView只遵守两个协议
- 11.缓存行高：estimatedHeightForRow不能和HeightForRow里面的layoutIfNeeded同时存在，这两者同时存在才会出现“窜动”的bug。所以我的建议是：只要是固定行高就写预估行高来减少行高调用次数提升性能。如果是动态行高就不要写预估方法了，用一个行高的缓存字典来减少代码的调用次数即可
- 12.不要做多余的绘制工作。在实现drawRect:的时候，它的rect参数就是需要绘制的区域，这个区域之外的不需要进行绘制。例如上例中，就可以用CGRectIntersectsRect、CGRectIntersection或CGRectContainsRect判断是否需要绘制image和text，然后再调用绘制方法。
- 13.预渲染图像。当新的图像出现时，仍然会有短暂的停顿现象。解决的办法就是在bitmap context里先将其画一遍，导出成UIImage对象，然后再绘制到屏幕；
- 14.使用正确的数据结构来存储数据。

2.如何提升 tableview 的流畅度？

- 本质上是降低 CPU、GPU 的工作，从这两个大的方面去提升性能。

CPU：对象的创建和销毁、对象属性的调整、布局计算、文本的计算和排版、图片的格

式转换和解码、图像的绘制

GPU：纹理的渲染

- 卡顿优化在 **CPU** 层面

尽量用轻量级的对象，比如用不到事件处理的地方，可以考虑使用 CALayer 取代 UIView

不要频繁地调用 UIView 的相关属性，比如 frame、bounds、transform 等属性，尽量减少不必要的修改

尽量提前计算好布局，在有需要时一次性调整对应的属性，不要多次修改属性

Autolayout 会比直接设置 frame 消耗更多的 CPU 资源

图片的 size 最好刚好跟 UIImageView 的 size 保持一致

控制一下线程的最大并发数量

尽量把耗时的操作放到子线程

文本处理（尺寸计算、绘制）

图片处理（解码、绘制）

- 卡顿优化在 **GPU**层面

尽量避免短时间内大量图片的显示，尽可能将多张图片合成一张进行显示

GPU能处理的最大纹理尺寸是 4096x4096，一旦超过这个尺寸，就会占用 CPU 资源进行处理，所以纹理尽量不要超过这个尺寸

尽量减少视图数量和层次

减少透明的视图（ $\alpha < 1$ ），不透明的就设置 `opaque` 为 YES

尽量避免出现离屏渲染

• iOS 保持界面流畅的技巧

1. 预排版，提前计算

在接收到服务端返回的数据后，尽量将 CoreText 排版的结果、单个控件的高度、cell 整体的高度提前计算好，将其存储在模型的属性中。需要使用时，直接从模型中往外取，避免了计算的过程。

尽量少用 UILabel，可以使用 CALayer。避免使用 AutoLayout 的自动布局技术，采取纯代码的方式

2. 预渲染，提前绘制

例如圆形的图标可以提前在，在接收到网络返回数据时，在后台线程进行处理，直接存储在模型数据里，回到主线程后直接调用就可以了

避免使用 CALayer 的 Border、corner、shadow、mask 等技术，这些都会触发离屏渲染。

3. 异步绘制

4. 全局并发线程

5. 高效的图片异步加载

3.APP启动时间应从哪些方面优化？

App启动时间可以通过xcode提供的工具来度量，在Xcode的Product->Scheme-->Edit Scheme->Run->Auguments中，将环境变量DYLD_PRINT_STATISTICS设为YES，优化需以下方面入手

- dylib loading time

核心思想是减少dylibs的引用

合并现有的dylibs（最好是6个以内）

使用静态库

- rebase/binding time

核心思想是减少DATA块内的指针

减少Object C元数据量，减少Objc类数量，减少实例变量和函数（与面向对象设计思想冲突）

减少c++虚函数

多使用Swift结构体（推荐使用swift）

- ObjC setup time

核心思想同上，这部分内容基本上在上一阶段优化过后就不会太过耗时

initializer time

- 使用initialize替代load方法

减少使用c/c++的attribute((constructor))；推荐使用dispatch_once() pthread_once() std:once()等方法

推荐使用swift

不要在初始化中调用dlopen()方法，因为加载过程是单线程，无锁，如果调用dlopen则会变成多线程，会开启锁的消耗，同时有可能死锁

不要在初始化中创建线程

4.如何降低APP包的大小

降低包大小需要从两方面着手

- 可执行文件

编译器优化：Strip Linked Product、Make Strings Read-Only、Symbols Hidden by Default 设置为 YES，去掉异常支持，Enable C++ Exceptions、Enable Objective-C Exceptions 设置为 NO， Other C Flags 添加 -fno-exceptions 利用 AppCode 检测未使用的代码：菜单栏 -> Code -> Inspect Code

编写LLVM插件检测出重复代码、未被调用的代码

- 资源（图片、音频、视频 等）

优化的方式可以对资源进行无损的压缩

去除没有用到的资源：<https://github.com/tinymind/LSUnusedResources>

6.怎么检测图层混合

1、模拟器debug中color blended layers红色区域表示图层发生了混合

2、Instrument-选中Core Animation-勾选Color Blended Layers

避免图层混合：

- 确保控件的opaque属性设置为true，确保backgroundColor和父视图颜色一致且不透明
- 如无特殊需要，不要设置低于1的alpha值
- 确保UIImage没有alpha通道

UILabel图层混合解决方法：

iOS8以后设置背景色为非透明色并且设置`label.layer.masksToBounds=YES`让label只会渲染她的实际size区域，就能解决UILabel的图层混合问题

iOS8 之前只要设置背景色为非透明的就行

为什么设置了背景色但是在iOS8上仍然出现了图层混合呢？

UILabel在iOS8前后的变化，在iOS8以前，UILabel使用的是CALayer作为底图层，而在iOS8开始，UILabel的底图层变成了_UILayer，绘制文本也有所改变。

在背景色的四周多了一圈透明的边，而这一圈透明的边明显超出了图层的矩形区域，设置图层的`masksToBounds`为YES时，图层将会沿着Bounds进行裁剪 图层混合问题解决了

7.日常如何检查内存泄露？

目前我知道的方式有以下几种

- Memory Leaks
- Allocations
- Analyse
- Debug Memory Graph
- MLeaksFinder

泄露的内存主要有以下两种：

- Leak Memory 这种是忘记 Release 操作所泄露的内存。
- Abandon Memory 这种是循环引用，无法释放掉的内存。

8.如何优化`APP`的电量？

- 程序的耗电主要在以下四个方面：

CPU 处理

定位

网络

图像

- 优化的途径主要体现在以下几个方面：

尽可能降低 CPU、GPU 的功耗。

尽量少用 定时器。

优化 I/O 操作。

不要频繁写入小数据，而是积攒到一定数量再写入

读写大量的数据可以使用 Dispatch_io，GCD 内部已经做了优化。

数据量比较大时，建议使用数据库

- 网络方面的优化

减少压缩网络数据（XML -> JSON -> ProtoBuf），如果可能建议使用 ProtoBuf。

如果请求的返回数据相同，可以使用 NSCache 进行缓存

使用断点续传，避免因网络失败后要重新下载。

网络不可用的时候，不尝试进行网络请求

长时间的网络请求，要提供可以取消的操作

采取批量传输。下载视频流的时候，尽量一大块一大块的进行下载，广告可以一次下载多个

- 定位层面的优化

如果只是需要快速确定用户位置，最好用 CLLocationManager 的 requestLocation 方法。定位完成后，会自动让定位硬件断电

如果不是导航应用，尽量不要实时更新位置，定位完毕就关掉定位服务

尽量降低定位精度，比如尽量不要使用精度最高的 kCLLocationAccuracyBest

需要后台定位时，尽量设置 `pausesLocationUpdatesAutomatically` 为 YES，如果用户不太可能移动的时候系统会自动暂停位置更新

尽量不要使用 `startMonitoringSignificantLocationChanges`，优先考虑 `startMonitoringForRegion`:

- 硬件检测优化

用户移动、摇晃、倾斜设备时，会产生动作(motion)事件，这些事件由加速度计、陀螺仪、磁力计等硬件检测。在不需要检测的场合，应该及时关闭这些硬件

部分总结的组件化+性能优化面试题就到这里了，
以上内容对你有帮助的话，记得关注我们！

文章来源于网络，已尽可能标明作者以及来源，文章内容为作者独立观点，不代表本公众号立场，因文章侵权本公众号不承担任何法律及连带责任，如有侵权，请联系我们删除。



觉得不错的话，别忘了点个"在看"哦~ 📌

收录于话题 #BAT大厂面试题合集·9个

上一篇

iOS面试题合集（网络篇）

下一篇

iOS面试题合集（架构+设计模式篇）

[阅读原文](#)

喜欢此内容的人还喜欢

好消息：在外地看门诊，社保也能直接报销了

定投十年赚十倍

大股东后院失火

诗与星空

猜！哪个指数回血最快？

养基之前